

Viernes, 28 de Agosto del 2026



PROYECTO INTEGRADOR DE REDES NEURONALES

Mini-JARVIS: asistente de voz en español

sobre la arquitectura Transformer

Britany Tahis Macias Tapuy

Redes Neuronales

Índice

1. Antecedentes.....	3
1.1. Objetivos Principales.....	4
1.2. Herramientas Utilizadas	4
2. Proyecto Realizado (Mini-JARVIS: asistente de voz en español)	5
2.1. La arquitectura Transformer y el módulo de exploración	5
2.2. Del Transformer al modelo de lenguaje	8
2.3. El recorrido de la voz: transcripción, modelo y síntesis.....	9
2.4. La interfaz y los estados del asistente.....	10
2.5. Herramientas y funcionamiento sin internet.....	12
2.6. Decisiones de diseño	13
2.7. Limitaciones conocidas	13
2.8. Pruebas realizadas	14
3. Conclusiones y Recomendaciones	14
4. Referencias	16

1. Antecedentes

Este proyecto consiste en un asistente conversacional por voz en español, al que denominé Elena. El usuario mantiene pulsado un botón y habla; el sistema transcribe el audio, envía el texto a un modelo de lenguaje preentrenado junto con el contexto de la conversación, recibe una respuesta escrita, la convierte en audio y la reproduce. Todo ocurre dentro de una ventana que indica en cada momento la etapa en la que se encuentra el sistema.

El trabajo articula dos planos. Por un lado la teoría de la arquitectura Transformer y de los modelos de lenguaje, estudiada en la asignatura. Por otro la integración de un modelo real dentro de una aplicación estable, que funcione de extremo a extremo y resuelva los fallos previsibles sin interrumpirse. No entreno ningún modelo, porque el enunciado no lo requiere y no resulta viable en el tiempo de un curso. En cambio sí inspeccioné uno internamente, con el fin de poder explicar el tratamiento que recibe una frase desde que ingresa hasta que se produce la respuesta.

El código fuente reside en un repositorio público de GitHub con 81 commits distribuidos por tareas, de modo que el historial refleja el avance y no un único envío final. De forma

complementaria mantengo una carpeta de control interno donde quedan registrados el plan de trabajo, las pruebas y las decisiones adoptadas, incluidas las alternativas descartadas.

1.1. Objetivos Principales

- Explicar y evidenciar, a partir de la salida real del propio programa, los conceptos de tokenización, embeddings, positional encoding y self-attention.
- Integrar un modelo de lenguaje preentrenado en una aplicación funcional de extremo a extremo, tanto mediante API en la nube como en ejecución local.
- Implementar el pipeline de voz completo: reconocimiento de voz, procesamiento con el modelo y síntesis de voz.
- Diseñar una interfaz que comunique de forma inequívoca el estado del asistente, y aplicar buenas prácticas de ingeniería de software: manejo de errores, gestión segura de credenciales, control de versiones y documentación técnica.
- Analizar las limitaciones, los sesgos y las implicaciones éticas del uso de un modelo de lenguaje en una aplicación destinada a terceros.

1.2. Herramientas Utilizadas

El proyecto está desarrollado en Python 3.14 sobre un entorno virtual, con todas las dependencias fijadas por versión para garantizar que una instalación en otra máquina reproduzca el mismo entorno.

El reconocimiento de voz emplea Whisper large-v3 (Radford et al., 2023), servido por Together AI. El procesamiento del lenguaje recae en dos modelos intercambiables desde la propia interfaz: Qwen3.8-2.4T-A95B como modelo principal y Llama-3.3-70B-Instruct-Turbo como

alternativo. La síntesis de voz utiliza edge-tts, cliente del servicio de síntesis de Microsoft, con la voz es-AR-ElenaNeural.

Para el funcionamiento sin conexión, esas tres piezas disponen de equivalentes que se ejecutan en la máquina local: faster-whisper para la transcripción, un modelo Qwen2.5 de 0.5B para la generación de respuestas y la voz del sistema operativo para la síntesis.

El módulo de exploración utiliza BETO (Cañete et al., 2020), un modelo de aproximadamente 110 millones de parámetros entrenado únicamente en español, cargado mediante Hugging Face Transformers (s.f.) y PyTorch, con matplotlib para la representación gráfica. La interfaz esta construida con CustomTkinter (Schimansky, s.f.). Las herramientas del asistente se apoyan en psutil para consultar el estado del equipo, ddgs para las búsquedas web y httpx para la consulta meteorologica. Las pruebas automáticas se ejecutan con pytest.

Conviene justificar una eleccion que puede resultar llamativa: la comunicacion con Together AI se realiza mediante la libreria de OpenAI. Together implementa una API compatible, de manera que basta con modificar la direccion del servidor. Desarrollar un cliente propio habria supuesto un esfuerzo adicional sin ninguna ventaja técnica.

2. Proyecto Realizado (Mini-JARVIS: asistente de voz en español)

2.1. La arquitectura Transformer y el módulo de exploración

Un modelo de lenguaje no procesa letras. La primera operación que se aplica a una frase es la tokenizacion: el texto se divide en unidades discretas denominadas tokens, que pueden corresponder a una palabra completa, a un fragmento de palabra o a un signo de puntuación, y cada unidad se sustituye por un identificador numerico. Las palabras ausentes del vocabulario del

modelo se fragmentan en varias piezas. En el módulo de exploración, la palabra Wikipedia se descompone en seis tokens.

Cada token se transforma a continuación en un vector de alta dimensionalidad, denominado embedding, que constituye la representación del significado con la que opera el modelo. Ese vector, sin embargo, no contiene información sobre la posición que la palabra ocupaba en la frase, y el mecanismo de atención tampoco la deduce, puesto que no procesa la secuencia en orden. Esa carencia la resuelve el positional encoding, una señal dependiente de la posición que se suma a cada token.

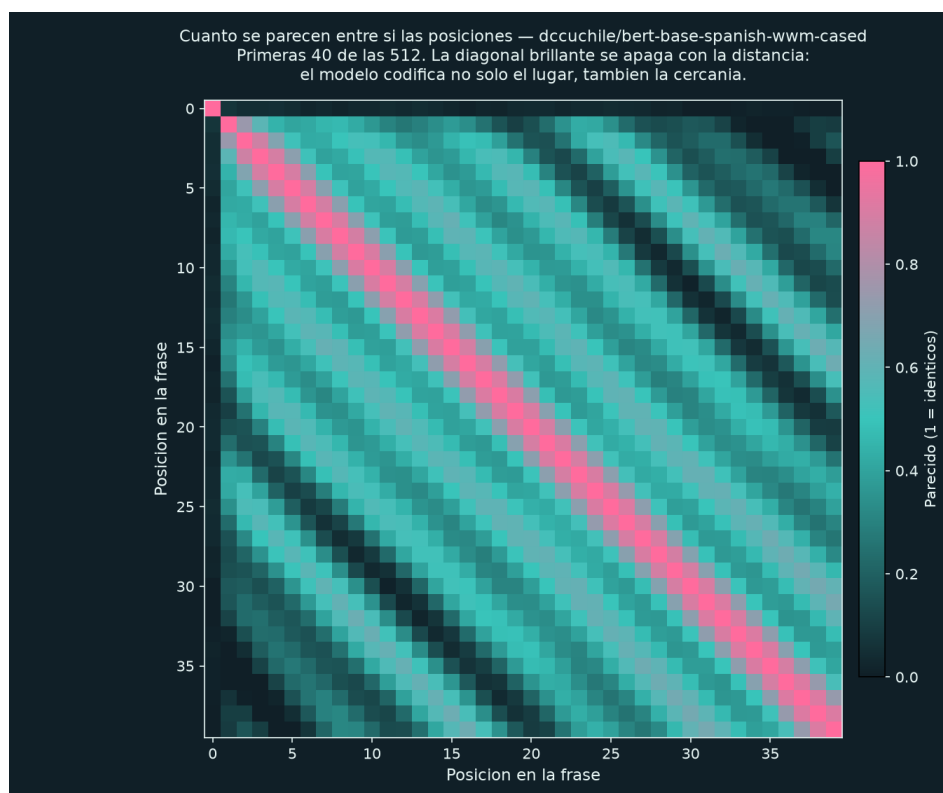


imagen 1 Similitud entre posiciones. La diagonal se atenúa con la distancia, lo que indica que el modelo codifica tanto el lugar como la proximidad.

Sobre esa representación opera el self-attention, mecanismo central de la arquitectura Transformer (Vaswani et al., 2017). Cada token pondera su relación con todos los demás de la

secuencia y construye así una representación contextual. De ello se deriva que una misma palabra adquiera significados distintos según el contexto: banco de río y banco de la calle comparten grafía, y lo que las diferencia es el conjunto de palabras al que cada una atiende. En la representación gráfica, cada fila corresponde a un token y la intensidad indica el peso asignado a los demás. Cada fila suma exactamente uno, ya que constituye una distribución de probabilidad.

Esta operación no se ejecuta una sola vez. El mecanismo se denomina multi-head attention porque el modelo realiza varias lecturas paralelas de la misma secuencia, y cada cabeza aprende a atender a un tipo distinto de relación lingüística: la concordancia entre sujeto y verbo, la referencia de un adjetivo a su sustantivo, entre otras. El modelo empleado en el módulo de exploración cuenta con doce capas de doce cabezas cada una, lo que produce ciento cuarenta y cuatro mapas de atención distintos para una misma frase, seleccionables desde la interfaz.

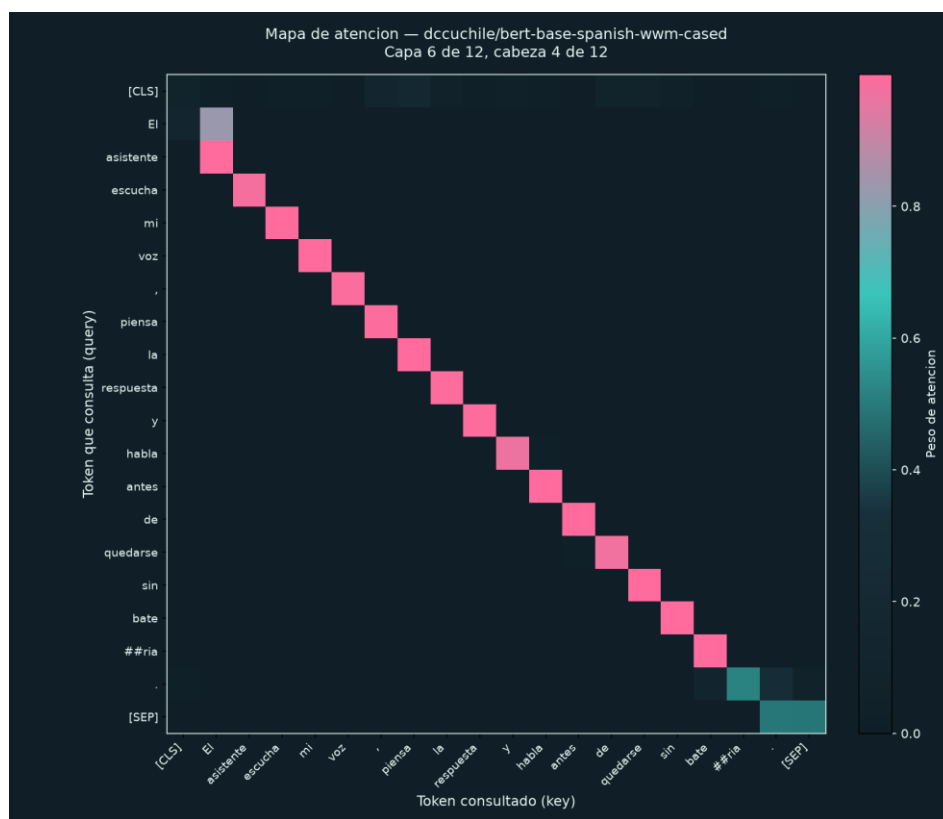


imagen 2 Mapa de self-attention correspondiente a una frase de ejemplo.

El módulo de exploración no emplea el mismo modelo que la conversación, y la razón es técnica. El modelo con el que dialoga Elena pertenece a un proveedor externo y su acceso se limita a una API que devuelve texto, por lo que no existe forma de obtener sus pesos de atención. BETO, en cambio, se descarga y puede inspeccionarse.

Durante el desarrollo se presentó un problema que estuvo cerca de invalidar el módulo completo. Al solicitar los pesos de atención, el resultado devuelto era vacío y no se generaba ninguna excepción. La causa reside en que la implementación de atención que el modelo carga por defecto no materializa esos pesos, y es necesario solicitar explícitamente la implementación clásica. Dado que el fallo era silencioso, el código incorpora ahora una comprobación que verifica la existencia de los pesos y notifica el error si vuelven a llegar vacíos.

2.2. Del Transformer al modelo de lenguaje

La arquitectura Transformer original constaba de dos componentes. Un encoder, que procesa la secuencia completa y construye su representación atendiendo simultáneamente a ambos lados de cada token, y un decoder, que genera texto de forma secuencial y solo puede atender a lo ya producido. Este proyecto emplea ambos. El módulo de exploración utiliza BETO, de tipo encoder-only, orientado a la comprensión, motivo por el cual sus mapas de atención resultan interpretables en ambas direcciones. La conversación, en cambio, recae en un modelo decoder-only autoregresivo, arquitectura predominante en los asistentes conversacionales actuales, dado que responder exige generar texto y no únicamente interpretarlo.

La generación autoregresiva consiste en producir un token, incorporarlo a la propia entrada y generar el siguiente a partir del conjunto resultante. Tres parámetros gobiernan esa selección. La

temperatura regula el grado de aleatoriedad: valores proximos a cero conducen a elegir siempre el token más probable, con respuestas predecibles, mientras que valores altos amplian el espacio de eleccion y aumentan tanto la originalidad como la probabilidad de error. El top_p opera de forma complementaria, restringiendo la selección a los candidatos más probables hasta acumular un porcentaje determinado. El maximo de tokens limita la extension de la respuesta, parámetro relevante en este proyecto porque toda respuesta se reproduce en voz alta.

Un modelo de lenguaje atraviesa varias etapas antes de poder conversar. El preentrenamiento lo expone a grandes volúmenes de texto y le enseña únicamente a continuar una secuencia. Un modelo que se detiene en esa fase se denomina modelo base y, ante una pregunta, tiende a prolongar el texto en lugar de responderlo. El fine-tuning consiste en continuar el entrenamiento con datos de un dominio concreto para especializarlo. El instruction-tuning es una variante de esa etapa que resulta determinante en este caso: enseña al modelo que los enunciados con forma de pregunta o de instruccion deben responderse. Los modelos empleados incorporan el termino Instruct en su identificador precisamente por haber atravesado esa fase. Sin ella el system prompt careceria de efecto, ya que un modelo base no obedece instrucciones sino que las continua.

El system prompt es el único elemento que define la personalidad del asistente. Establece su nombre, su condicion de sistema automático y el estilo de redaccion exigido: frases breves, sin listas ni simbolos de formato, dado que el texto se convierte en audio.

2.3. El recorrido de la voz: transcripción, modelo y síntesis

El código está organizado por responsabilidades. Un paquete para el núcleo, que agrupa la captura de audio, la transcripción, el motor de lenguaje, la memoria conversacional y el orquestador encargado de coordinar cada turno. Otro para la interfaz gráfica, otro para las herramientas y otro

para el módulo de exploración. Un único archivo de configuración centraliza todos los parámetros ajustables.

La grabación se produce mientras el botón permanece pulsado y, al liberarlo, el audio se envía a transcribir. La grabación no se escribe en disco en ningún momento: permanece en memoria, se transcribe y se descarta.

El texto transcrito se envía al modelo acompañado del system prompt y de los últimos turnos de la conversación. En este punto interviene la ventana de contexto, que delimita la cantidad máxima de texto que el modelo puede procesar en una sola petición. El modelo no conserva información entre llamadas sucesivas; la continuidad aparente de la conversación se obtiene reenviando el historial completo en cada turno. Puesto que ese historial no puede crecer indefinidamente sin exceder la ventana ni incrementar el coste, el sistema conserva los diez últimos turnos y descarta el más antiguo. Si la conversación se prolonga, el asistente no falla: únicamente pierde la información inicial, y un indicador en la interfaz advierte de ello antes de que ocurra.

La respuesta generada se convierte en audio y se reproduce. El estilo breve impuesto por el system prompt responde a esta última etapa, ya que los elementos de formato carecen de equivalente hablado.

2.4. La interfaz y los estados del asistente

La ventana se organiza en dos columnas. La izquierda contiene un indicador circular que varía según el estado del sistema; la derecha, la transcripción de la conversación y los controles. Los estados son cuatro, más el de error, y cada uno modifica el color y el movimiento del indicador, de manera que la etapa en curso resulta identificable sin necesidad de leer.

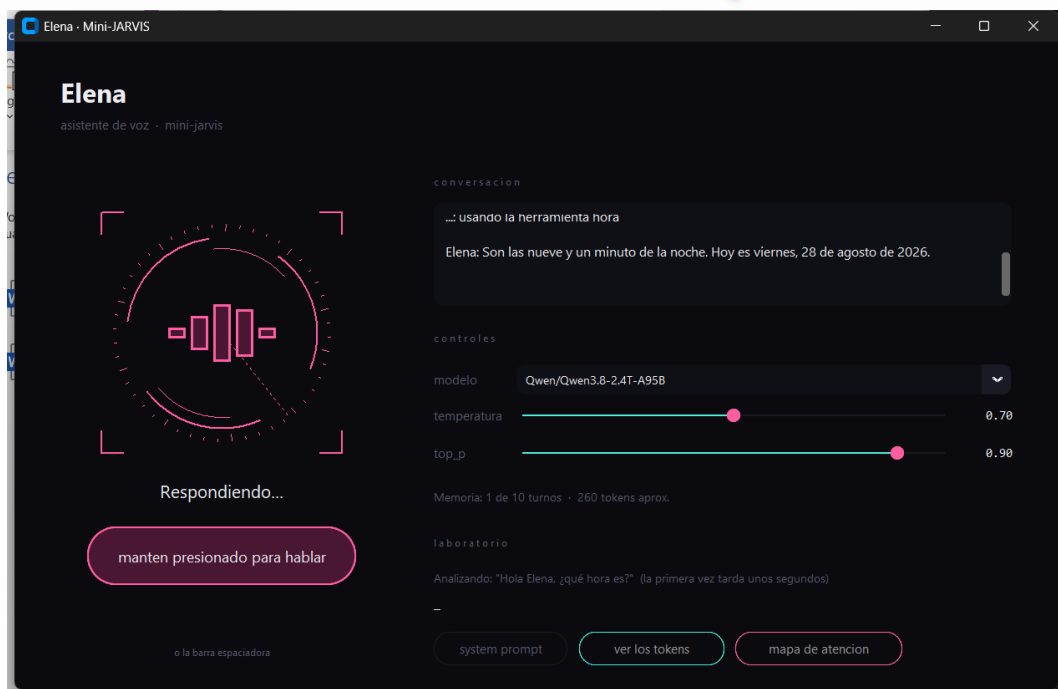


imagen 3 La aplicación durante un turno real. El indicador muestra el estado Respondiendo, la conversación registra el uso de una herramienta y el contador senala un turno de los diez que caben en memoria.

Bajo la conversación se disponen controles orientados a la demostración de los conceptos teóricos: un selector de modelo, deslizadores de temperatura y top_p, y un contador de memoria que advierte cuando el turno más antiguo va a descartarse. El deslizador de temperatura alcanza un máximo de 1.4 y no de 2 por una razón medida: en las pruebas contra la API, a partir de 1.5 el modelo de razonamiento se bloqueaba durante aproximadamente cien segundos en dos de cada tres intentos.

Los fallos previsibles están contemplados y ninguno interrumpe la ejecución: ausencia de conexión, audio no reconocido, respuesta vacía, credencial incorrecta y tiempo de respuesta excesivo. Cada situación dispone de un mensaje específico, dado que un mensaje que atribuye el fallo a una causa equivocada resulta más perjudicial que la ausencia de mensaje. En cuanto a accesibilidad, la interacción admite también la barra espaciadora además del botón, y el estado nunca se comunica únicamente mediante color: siempre lo acompaña una indicación textual.

2.5. Herramientas y funcionamiento sin internet

El asistente no se limita a conversar, sino que ejecuta acciones sobre el sistema mediante tool calling. El modelo recibe la relación de operaciones disponibles y, cuando la petición lo requiere, responde solicitando una de ellas en lugar de generar texto. El sistema dispone de diez: cálculo aritmético, consulta de hora, consulta meteorológica, estado del equipo, búsqueda web, apertura de páginas, control de volumen, control de brillo, apertura de carpetas y reproducción de video.

Toda operación que actúa sobre el sistema atraviesa previamente una lista blanca. La calculadora, por ejemplo, no recurre a la función de evaluación de Python, que ejecutaría cualquier expresión generada por el modelo; en su lugar analiza sintácticamente la expresión y admite exclusivamente operaciones aritméticas. Las páginas y las carpetas accesibles proceden igualmente de una relación cerrada. El modelo propone la acción, pero la decisión sobre su ejecución corresponde al programa.

Aunque el enunciado no lo exige, el asistente conserva su funcionalidad sin conexión. Ante una caída de red, las tres piezas del pipeline conmutan automáticamente a sus equivalentes locales y la interfaz señala el cambio de modo. La calidad de las respuestas disminuye de forma apreciable, dado que el modelo local es aproximadamente mil veces menor que el remoto. A cambio, el sistema sigue operativo, mientras que sin este mecanismo una pérdida de conexión lo inutilizaba por completo.

La conmutación distingue entre una caída de red y una credencial mal configurada. En el segundo caso el sistema no cambia a modo local, puesto que hacerlo ocultaría un error de configuración en lugar de exponerlo.

2.6. Decisiones de diseño

Cada turno de conversación se ejecuta en un hilo independiente que se crea y finaliza con el propio turno. La alternativa consistía en construir el sistema sobre asincio, opción técnicamente más elegante, pero la biblioteca de interfaz gráfica no es asincrónica y su combinación exige un puente que introduce fallos difíciles de reproducir. Un hilo por turno resulta más simple de comprender y de justificar.

La elección de dos modelos de familias distintas, en lugar de dos tamaños de la misma, responde a una incidencia real. A tres días de la entrega, el proveedor retiró el modelo alternativo configurado. La verificación de los 169 modelos del catálogo reveló que únicamente veinte respondían. Si ambos modelos pertenecen a la misma familia, una incidencia del proveedor puede afectarlos simultáneamente.

La memoria conversacional se trunca por turnos y no por tokens. El recuento exacto de tokens requiere el tokenizador del proveedor y depende de la conexión, mientras que el recuento por turnos puede probarse sin red y visualizarse en la interfaz. La decisión sacrifica precisión a cambio de un comportamiento verificable y demostrable.

2.7. Limitaciones conocidas

La limitación más relevante es la generación de contenido incorrecto. El modelo puede producir un dato falso y expresarlo con el mismo grado de seguridad que uno verificado. No se trata de un defecto del programa sino de una característica inherente al funcionamiento de los modelos de lenguaje, por lo que no admite corrección mediante código. La medida adoptada es informativa: el asistente advierte al iniciarse de su naturaleza automática y de la posibilidad de error, y la misma advertencia figura en su system prompt.

El rendimiento sin conexión es sensiblemente inferior. El modo con conexión depende de la disponibilidad de saldo en la API y de la latencia de red, de modo que una respuesta puede demorarse varios segundos. El modelo principal es cerrado: no es posible examinar sus pesos ni conocer los datos con los que fue entrenado, lo que impide auditar sus sesgos. La transcripción pierde fiabilidad en presencia de ruido ambiental o cuando la distancia al microfono es excesiva, y en tales casos el modelo responde a un enunciado que el usuario no formulo. Por ultimo, el sistema no identifica a su interlocutor, por lo que no resulta apropiado para aplicaciones que requieran autenticación de personas.

2.8. Pruebas realizadas

El proyecto cuenta con 189 pruebas automáticas que se ejecutan en aproximadamente doce segundos sin requerir conexión, microfono ni consumo de la API. Cubren el análisis de las respuestas del modelo, la memoria conversacional y su truncado, las diez herramientas, el tratamiento de cada tipo de error y la conmutacion a modo local.

Existe además un conjunto de verificaciones que no admiten automatizacion, por tratarse de comprobaciones perceptivas: la naturalidad de la voz sintetizada, la distinguibilidad de los estados en la interfaz y la aparicion del aviso de modo local al desactivar la conexión. Estas se realizaron manualmente y quedan registradas. Se verificaron asimismo los tres escenarios principales: una conversación de varios turnos consecutivos, una peticion que obliga al uso de una herramienta y una interrupcion de red durante el uso.

3. Conclusiones y Recomendaciones

El desarrollo confirma que es posible construir un asistente de voz completo sobre un modelo preentrenado sin necesidad de entrenar ninguno. La integracion de los componentes resulto

ser la parte menos exigente del trabajo; la comprensión del funcionamiento interno de cada uno requirió un esfuerzo considerablemente mayor, y es en ese punto donde el módulo de exploración aportó el mayor valor formativo, al obligar a examinar el modelo en lugar de tratarlo como una caja opaca.

La decisión de mayor alcance fue mantener el orquestador desacoplado del resto de componentes. Esa separación, adoptada inicialmente por claridad, permitió incorporar posteriormente el modo sin conexión sin modificar ninguna línea de la lógica conversacional: fue suficiente con envolver las tres piezas que se comunican con el servicio remoto.

En el plano ético, la conclusión es que no toda limitación de un sistema de este tipo admite solución técnica. Cuando el problema reside en el comportamiento del modelo y no en el código que lo rodea, la única respuesta responsable consiste en informar al usuario y documentarlo, en lugar de presentar el sistema como más fiable de lo que es.

A partir de la experiencia obtenida se formulan las siguientes recomendaciones.

- Incorporar una palabra de activación que evite la dependencia del botón. Es el único requisito opcional del enunciado que no llegó a cubrirse.
- Sustituir el modelo local por uno de mayor tamaño cuando las condiciones del equipo lo permitan, con el fin de reducir la diferencia de calidad entre ambos modos de funcionamiento.
- Verificar la disponibilidad de saldo en la API antes de cualquier demostración, dado que el modo con conexión depende de ella.
- Revisar periódicamente la vigencia de los modelos seleccionados, puesto que durante el desarrollo un proveedor retiró uno de ellos con tres días de antelación.

4. Referencias

Cañete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., y Perez, J. (2020). Spanish pre-trained BERT model and evaluation data. PML4DC at ICLR 2020.

<https://github.com/dccuchile/beto>

Hugging Face. (s.f.). Transformers documentation. Recuperado el 28 de agosto de 2026, de

<https://huggingface.co/docs/transformers>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., y Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning. <https://arxiv.org/abs/2212.04356>

rany2. (s.f.). edge-tts [Software]. GitHub. Recuperado el 28 de agosto de 2026, de

<https://github.com/rany2/edge-tts>

Schimansky, T. (s.f.). CustomTkinter documentation. Recuperado el 28 de agosto de 2026, de

<https://customtkinter.tomschimansky.com/>

SYSTRAN. (s.f.). faster-whisper [Software]. GitHub. Recuperado el 28 de agosto de 2026, de

<https://github.com/SYSTRAN/faster-whisper>

Together AI. (s.f.). Together AI documentation. Recuperado el 28 de agosto de 2026, de

<https://docs.together.ai/>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., y

Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information

Processing Systems, 30. <https://arxiv.org/abs/1706.03762>